Looping

Computers are really great at doing repetitive tasks. Hopefully by now you guys have realized that when we are programming, we want to do the least amount of work possible. Swift makes it possible to use ranges of numbers, and lists of items to do repetitive work easily, and allows us to write the least amount of code possible!

Let's start with something simple. Let's say we have a list of Albums.

```
let albums = ["The Search", "Clouds", "Therapy Session", "Perception",
"Mansion"]
```

Now let's say we want to print each of these album names with the text “is a NF Album”, how would we do that right now? Something like this right?

```
// There are 5 items so we need to print out indices
// 0 - 4.
print("\(albums[0]) is a NF Album")
print("\(albums[1]) is a NF Album")
print("\(albums[2]) is a NF Album")
print("\(albums[3]) is a NF Album")
print("\(albums[4]) is a NF Album")
```

This does work, but it's a mess, and what if the amount of albums changes? If we lost an album then our code would crash, and if we gained an album it wouldn't print unless we changed our code. Also, beyond that, this is a super repetitive task right? We are doing the same thing 4 times with just slightly different values.

So now that we have shown a way to do it without looping, let's do it with looping.

```
for album in albums {
    print("\(album) is a NF Album")
}
```

Let's look at this new Syntax and pick it apart.

1. `for` — keyword to say that we want to use a for loop / loop over something.
2. `album` — we reference this inside of that block of loop code, and it acts similar to a constant. Notice, we never declared a variable or constant named `album`.
3. `in` — saying where we are getting the information from.
4. `albums` — the name of the array we are accessing.

Putting it all together makes a very easy to read statement, right? We are saying that we want to do something for each album in our list of albums.

What if we tried to reference `album` outside the `for loop` code? HINT: it doesn't work. We talked about how the `{}` curly braces represent a section of code previously. But now this is a different context, but they have the same general meaning. The `{}` curly braces contain something, the code inside the `{}` curly braces can only be accessed from other code between the `{}` curly braces. You can, however, access code that is outside the curly braces from within.

```
var numberOfItems = 0

// Warning: `album` is never used.
for album in albums {
    numberOfItems += 1
}

print("There are \(numberOfItems) albums.")
```

Strangely, we have a yellow warning in our code. (This may have been removed recently, I cannot find any patch notes regarding its removal, but it does not show for me anymore). This warning is telling us that we have this constant we created named `album` but we never use it. As you can see, we never reference album in our block of code. So we are creating this constant and making a spot in memory for it, for no reason. We don't want to do that, so there is a standard procedure that we have already talked about. We can replace the name of the constant with an `_` to let Swift know that we don't care about creating it.

You will see this syntax used with `for loops` a lot, so let's go and rewrite the above example with the proper syntax and get rid of that warning.

```
numberOfItems = 0
for _ in albums {
    numberOfItems += 1
}

print("There are \(numberOfItems) albums.")
```

When we are iterating over `Arrays` and `Sets` it is the exact same syntax, just with `Sets` remember that the order will be different each time you iterate through them.

We cannot iterate through `Tuples`, we can iterate through `Dictionaries` the syntax is different as we are iterating through `Keys` and `Values` instead of just a `Value` so let's do an example for that.

```
var artistsAndAlbums = [
    "NF": albums,
    "Post Malone": ["Stoney", "Twelve Carat Toothache", "Hollywood's
Bleeding"]
]

for (artist, albums) in artistsAndAlbums {
    print("\(artist) has these albums: \(albums)")
}
```

We can also choose to just do the `Keys` or `Values` by replacing their names with an `_` like before.

```
// Only Values
for (_, albums) in artistsAndAlbums {
    print(albums)
}

// Only Keys
for (artist, _) in artistsAndAlbums {
    print(artist)
}
```

There are other ways that we can do what we are doing above, but this lesson is about `for loops` so we are focusing on that.

We are not always going to be iterating through a `Collection` sometimes we want to do something a set number of times, we can do that by using a `Range`. Let's do an example of printing the numbers 1 through 3.

```
for i in 1...3 {
    print(i)
}
```

This will print out 1, 2, 3 each on their own line. We can also again choose to ignore the number if we want, I would say most of the time I am doing a `Range for loop` I am ignoring the number.

```
// Prints "Hello" 4 times.
for _ in 1...4 {
    print("Hello")
}
```

We can also use what are called non-inclusive ranges. Which simply means not including the final number. So we could rewrite the above like this:

```
// Prints "Hello 2" 4 times
for _ in 0..<4 {
    print("Hello 2")
}
```

There is one last `for loop` declaration I want to show off, it may not seem super useful at first, but I promise it is. Sometimes instead of looping over the items within an `Array` directly, we want to iterate over the `Indices`. How could we do that? By using the `Range` syntax!

```
let someArrayOfStrings = ["StringOne", "StringTwo", "StringThree"]

for index in 0..<someArrayOfStrings.count {
    // We can use the index to access the value!
    print(someArrayOfStrings[index])
}
```

It is important that we use the non-inclusive range here, as if you remember, our count is one higher than whatever our highest index is.

While Loops

There are situations where we aren't going to know how many times we need to loop, or times we want to loop until a certain condition is met. This is where what are called `while loops` come in.

`While loops` take a condition and a block of code, like an `if` statement, and runs that block of code on repeat until the condition is `false`.

```
var number = 10

while number > 0 {
    print("The number is \(number)")
    number -= 1
}
```

`While loops` do introduce a rather scary thing, that we always need to be careful to avoid. Infinite loops... An infinite loop is exactly what it sounds like, it's a loop that will never end and will just keep running forever. This will halt all your code, and will lead to

your program crashing as it takes too much power to run an infinite loop. Let's do an example of an infinite loop, so that you can see how it's something that's easy to miss.

```
var infiniteLoopNumber = 0
while infiniteLoopNumber ≤ 10 {
    print(infiniteLoopNumber)
    infiniteLoopNumber -= 1
}
```

As you can see in the above loop, `infiniteLoopNumber` can never be above 10 with the code we have, so this `while loop` can never end. So something as simple as having a `-` instead of a `+` can be hazardous to your code.

I'm going to have you guys actually comment out the infinite `while loop` so you don't accidentally crash your computers by running it.

Repeat Loops

There is a third type of loop, that isn't used as much as the other ones, but it is really easy to learn so we are going to go over it. It is called a `repeat loop`. A `repeat loop` is essentially a while loop, the difference being that the condition is evaluated after the block of code is ran. So it will always run at least once.

```
repeat {
    print("This statement is false")
} while false
```

Looking at that syntax you would think that it would never run, because you are putting `false` as its condition. However, it runs a single time, because it doesnt know it's false until after it has ran.

Skipping Items

Break

Inside of loops, you can run essentially any code that you want to. Other loops, `if` statements etc. You may have an array of items that you want to print out, but after you reach a certain item, you don't want to run your loop anymore. Let's try to write that code.

```
var characters = ["Finn", "Jake", "The Lich", "BMO"]

for character in characters {
    if character == "The Lich" {
        break
```

```
    }

    print("\(character) is a Good Guy!")
}
```

Here I introduced a small piece of code that you haven't seen before. The syntax I am referring to is the `break` keyword. `break` has a couple of different uses depending on the context it is being used for. When we use break inside a loop we are saying that I want to stop running the code of the loop I am in, and I want to stop looping. So notice how if we run the code above, it won't print `"BMO"` even though `"BMO"` is not equal to `"The Lich"` it doesn't print his name. This is because `break` ends the loop, so since `"BMO"` appears after `"The Lich"` in the `Array`, he also doesn't get printed.

`Breaks` work in all the different loop types, in fact they can help us get out of an infinite loop.

```
var counter = 0

while true { // Typically while true would never end.
    print("Counter = \(counter)")
    counter += 1

    if counter > 10 {
        break // ends the infinite loop.
    }
}
```

Continue

There is another piece of syntax that is similar to `break`, the difference being that it stops running the code for the current iteration, but doesn't end the loop. So if I wanted to print every item except a specific one, I would use this syntax.

```
for char in characters {
    if char == "The Lich" {
        continue
    }

    print("\(char) is a Great Guy!")
}
```

Let's do a more complicated example that uses both a `continue` and a `break`. We are given a sorted `Array` of numbers, we need to create a new `Array` that contains all the

numbers that aren't a multiple of 3, and are less than 15. This is a more complicated problem than the ones we have dealt with before, so lets try to break it down into some pieces before we start coding. When we are facing a hard problem its easiest to break it down into some pieces and then tackle those pieces one at a time. I like to do this by writing comments. So let's do that:

```
// Using a given Sorted Array of Numbers
// Create a New Array that contains the numbers that are not multiples
of 3
// The Array should also not contain numbers that are 15 or higher.

// Create an Array to store the numbers.
// Go through all the numbers in the Given Array
// Check if its a multiple of 3
// Add the number to our Array if it is.
// Check if the number is 15 or higher
// if It is stop going through the numbers as all the numbers will be
too high after that
```

Now that we have broken down the problem, the next step is to write what is called Pseudocode. Pseudocode is a fancy way to say writing code but as English. So instead of jumping head first into code we can write out what steps we need to take in a more readable format. And again, comments are a great way to do this.

```
// variable chosenNumbers

// Loop through each of the numbers in the given array.
    // if multilpe of 3
        // ignore this number.
    // if 15 or greater
        // Stop looping.

    // Add number to chosenNumbers
```

I like to use indentation to show how things may nest, but if that hard for you to understand then don't do it. Pseudocode is meant to be as understandable as possible for you, so everyone's will look different.

Now that we have our Pseudocode, it's time to move on to converting that to real code. Again we only need to tackle one thing at a time. So we can start by doing the things that we know, and then deal with the things that we don't.

```
var chosenNumbers = [Int]()

for num in givenArray {
    // Check if multiple of 3
    if true {
        // Stop running this iteration.
    }

    if num ≥ 15 {
        // Stop looping.
    }

    chosenNumbers.append(num)
}
```

Now that we have done the things we are very familiar with, we can move onto solving the problems that we might not have known how to solve off the top of our head. Just one by one.

```
for num in givenArray {
    if num % 3 == 0 {
        continue
    }

    if num ≥ 15 {
        break
    }

    chosenNumbers.append(num)
}
```

Nesting Loops

As I have said before, inside our `for loops`, `while loops`, `if statements`, etc. we can write whatever code we need to. We can have loops that contain other loops, this is actually super common. Let's go to one of our previous examples, and change it up to use loop nesting.

```
artistsAndAlbums = [
    "NF" : albums,
    "Post Malone" : ["Stoney", "Twelve Carat Toothache", "Hollywood's
Bleeding"]
```

```
]

for (artist, albums) in artistsAndAlbums {
    for album in albums {
        let stringToPrint = "\(artist) wrote \(album)"
        print(stringToPrint)
    }
}
```

Notice how we are referencing above our current loop here. When we are between `{}` we can access anything above us. So anything the parent scope as access to we can access. But the parent cannot access the child's code. So if I tried to access `stringToPrint` in the parent for loop, I would get an error.

Breaking in Nested Loops

When we use a `break` inside a nested loop, we only break out of the current loop. So if we have a `break` inside the child loop, the parent loop will continue looping. Let's do a quick, simple example to show this off.

```
for numOne in 0...3 {
    print(numOne)
    for numTwo in 3...6 {
        print(numTwo)
        if numTwo ≥ 5 {
            break
        }
    }
}
```

Labeled Statements

So now we have covered out to break out of a single loop, and we have covered how to nest loops. But what if we need to break out of multiple levels of looping? Swift has what are called `labeled statements` that are meant to handle this problem. They let you exit any number of loops or conditions so that execution picks up exactly where you want it to.

Let's say we have an `Array` of `Arrays` of `Integers`. In this grid of `Integers` there is a single number that is divisible by 4, our goal is to find the position of that number and print it. Since there is only a single number, we can stop looking once we find it.

```
let numberGrid = [
    [1, 1, 3],
    [3, 4, 5],
```

```
    [7, 9, 9]
]

parentLoop: for y in 0..<numberGrid.count {
    for x in 0..<numberGrid[y].count {
        let number = numberGrid[y][x]
        if number % 4 == 0 {
            print("Found number \(number) at index \(y), \(x)")
            break parentLoop
        }
    }
}
```

Overview

1. Loops let us repeat code until a condition is false. That can be an actual `Boolean` Condition or a situation where we run out of things to iterate through.
2. The most common loop is a `for loop`, which assigns each item inside of a `Range` or `Collection` to a temporary constant.
3. If you don't need the temporary constant that a `for loop` gives you, you can use an underscore instead so Swift can skip that work.
4. There are `while loops`, which provide an explicit condition to check, and will run until that condition is false.
5. Although similar to `while loops`, `repeat loops` always run the body of their loop at least once, as they check their condition after running their body of code.
6. You can exit a single loop using `break`, but if you have nested loops you need to use labeled statements with your `break` in order to exit multiple.
7. You can stop running a `loops` block of code while continuing to iterate further using the `continue` keyword.
8. Infinite loops can happen if you don't have a proper expression for your while loop, or if you are missing a way to `break` out of it.